

Day 3 — Regression Evaluation and Validation

Objective

Learn to decide whether a regression model is actually useful.

Topics and detailed notes

- MAE measures average absolute error and is easy to interpret in target units.
- MSE penalizes large errors more strongly.
- RMSE returns to target units while retaining MSE's sensitivity to large errors.
- R^2 compares explained variation against a mean-prediction baseline.
- Adjusted R^2 penalizes unnecessary predictors in a classical linear-model setting.
- Train/test split is a simple generalization estimate.
- Cross-validation provides multiple training/validation views of the training data.
- Validation is for decisions; the final test set is for the final unbiased estimate after decisions.

Mathematics / formulas to know

1. Linear score

Logistic regression first calculates:

$$z = w^T x + b$$

or:

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

2. Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Therefore:

$$0 < \sigma(z) < 1$$

The output is interpreted as a probability:

$$P(y = 1|x) = \sigma(w^T x + b)$$

3. Classification threshold

For the usual threshold:

$$\hat{y} = \begin{cases} 1 & p \geq 0.5 \\ 0 & p < 0.5 \end{cases}$$

But later you should learn why 0.5 isn't always the optimal threshold.

4. Odds

$$Odds = \frac{p}{1-p}$$

5. Log-odds / Logit

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

Logistic regression models the log-odds as a linear function:

$$\ln\left(\frac{p}{1-p}\right) = w^T x + b$$

This is a very important concept to understand rather than simply memorizing the sigmoid.

6. Binary Cross-Entropy / Log Loss

For one observation:

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

For n observations:

$$J = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

7. Gradient

For the weights:

$$\frac{\partial J}{\partial w} = \frac{1}{n} X^T (p - y)$$

For the bias:

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)$$

8. Gradient Descent

$$w := w - \alpha \frac{\partial J}{\partial w}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

Implementation pattern

Dataset → split → baseline → model → cross-validation → metric summary → residual/error analysis → final test once.

Experiments to perform

- Compare MAE, RMSE and R^2 on the same predictions.
- Create a model that has good R^2 but unacceptable large errors and investigate why.
- Run K-fold cross-validation and compare mean and spread of scores.

End-of-day deliverables

- Business revenue prediction project.
- Metric comparison table.
- Cross-validation results.
- Short recommendation explaining which metric matters and why.

Mastery questions

1. Which metric would you choose for a business forecasting problem and why?
2. Why should the test set not guide repeated model decisions?
3. What does a negative R^2 mean?

PLAN

Welcome to Day 3. Your exam results from Day 2 prove that you have officially graduated from basic math to system architecture. Now, we introduce a major plot twist in your curriculum.

While your syllabus title says "**Regression Evaluation and Validation**", the exact mathematical formulas provided in your Day 3 document are actually the engine for **Logistic Regression (Classification)**.

As your Senior ML Engineer, I see what this curriculum is doing. It is transitioning you from predicting continuous numbers (Linear Regression, like predicting game sales) to predicting discrete probabilities (Logistic Regression, like predicting if a user is "Studying" or "Distracted" on a webcam feed). We will master both concepts today.

Here is the exact master plan for Day 3, mapped to your Global ML Workflow.

Phase 1: The Theory of Validation & Classification

This phase dictates **Step 10 (Validate and compare)** and **Step 13 (Freeze the final test set until final evaluation)** of our workflow.

- **The Validation Layer (Cross-Validation):** A single train/test split is just a simple generalization estimate. Today, we implement **K-Fold Cross-Validation**, which provides multiple training/validation views of the training data, ensuring our model isn't just getting lucky on a specific data split.
- **The Classification Switch:**
 - Instead of predicting raw dollars, we predict a **Linear score (\$z\$)**: $z = w^T x + b$.
 - We pass that score through a **Sigmoid function**: $\sigma(z) = \frac{1}{1 + e^{-z}}$. This physically forces the output to exist strictly between 0 and 1 ($0 < \sigma(z) < 1$).
 - This output is interpreted as a **probability** ($P(y=1|x) = \sigma(w^T x + b)$). For example, "There is an 85% probability the user is studying."
- **Log-odds/Logit:** Logistic regression models the log-odds as a linear function ($\ln(\frac{p}{1-p}) = w^T x + b$). This is the true, hidden mathematics of classification.

Phase 2: The Mathematical Engine of Classification

This replaces our Mean Squared Error from yesterday. When predicting categories (0 or 1), squaring the error fails.

The Loss Function:

- **Binary Cross-Entropy / Log Loss:**
 $J = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$. This math heavily punishes the model if it confidently predicts "0" when the true answer is "1".

The Gradient (The Steering Wheel):

- For the weights: $\frac{\partial J}{\partial w} = \frac{1}{n} \sum_{i=1}^n X^T (p_i - y_i)$.
- For the bias: $\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)$.
- *Notice anything?* These gradients look almost exactly identical to the Linear Regression gradients from Day 2! The only difference is that p (probability) has replaced $\hat{y}$ (linear prediction).

The Decision Rule:

- **Classification threshold:** $\hat{y} = \begin{cases} 1 & p \geq 0.5 \\ 0 & p < 0.5 \end{cases}$. (If the probability is over 50%, we guess 1. Otherwise, 0).

Phase 3: Today's Tasks & Implementation Pattern

We will strictly follow this pattern: Dataset $\rightarrow$ split $\rightarrow$ baseline $\rightarrow$ model $\rightarrow$ **cross-validation** $\rightarrow$ metric summary $\rightarrow$ residual/error analysis $\rightarrow$ final test once.

Task 1: The Regression Evaluation & Cross-Validation Engine

- We will compare MAE, RMSE and R^2 on the same predictions.
- *Experiment*: Create a model that has good R^2 but unacceptable large errors and investigate why. (We will prove why relying on a single metric is dangerous).
- *Experiment*: Run K-fold cross-validation and compare mean and spread of scores.

Task 2: The Classification Engine (From Scratch)

- You will write the **Sigmoid function** and **Binary Cross-Entropy** formulas in pure Python/NumPy, proving you understand how a model predicts probabilities.

Task 3: Business Recommendation

- Write a short recommendation explaining which metric matters and why for a business forecasting project.

Phase 4: Final Deliverables

By the end of today, you will deliver a Business revenue prediction project containing:

1. A Cross-validation results table.
2. A Metric comparison table.
3. A short, written recommendation to the business stakeholders.

Are you ready to dive into **Phase 1: The Theory** so I can explain exactly *why* K-Fold Cross-Validation is the industry standard over a simple train/test split?